LAB 7 – FUZZING FOR INPUT VALIDATION BUGS

Lab Report

Name: Palleneni Sloka
SRN: PES1UG24AM183
Course: Software Engineering
Assigned repository: L6_L7_03_coffee_shop_order

1. Objective

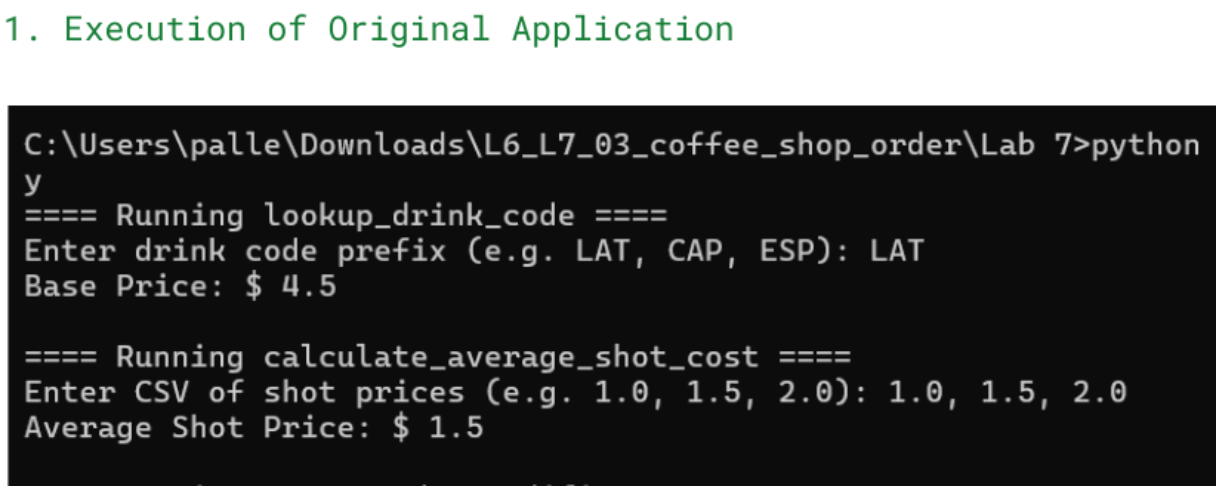
The objective of this lab was to use fuzz testing with the Hypothesis library to identify input-validation bugs in a Python application and improve the program so that it handles unexpected inputs safely.

2. Software Used

Python 3.11, Hypothesis 6.170.15 and Pytest 9.1.1.

3. Execution of Original Application

The original Coffee Shop Order application was run with normal sample inputs. The terminal output below shows the drink price, average shot price and parsed order modifiers.



4. Fuzz Testing – Failed Test Cases

The three test functions in test_processor.py were completed using Hypothesis. Generated inputs included empty strings and None. The original functions failed on these edge cases. The test summary reported three failed tests.

```
    | AssertionError: parse_order_modifiers crashed with: 'NoneType' object
has no attribute 'split'
    | assert False
    | Failing test case: test_parse_order_modifiers_safe(
    |     s=None,  # or any other generated value
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         C:\Users\palle\Downloads\L6_L7_03_coffee_shop_order\Lab 7\test
_processor.py:26
    +------------------------------------
========================= short test summary info =========================
FAILED test_processor.py::test_lookup_drink_code_no_crash - ExceptionGroup:
Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_processor.py::test_calculate_average_shot_cost_safe - ExceptionG
roup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_processor.py::test_parse_order_modifiers_safe - ExceptionGroup:
Hypothesis found 2 distinct failures. (2 sub-exceptions)
============================ 3 failed in 3.80s ============================

C:\Users\palle\Downloads\L6_L7_03_coffee_shop_order\Lab 7>

C:\Users\palle\Downloads\L6_L7_03_coffee_shop_order\Lab 7>
```

5. Changes Made

The following input checks and defensive handling were added to processor.py:

• **lookup_drink_code():** checks the input type, removes surrounding spaces, converts the code to uppercase and safely handles unknown drink codes.
• **calculate_average_shot_cost():** checks for missing or invalid CSV input and handles invalid, negative or non-finite prices.
• **parse_order_modifiers():** checks the input type and skips malformed entries rather than raising an error.

6. Execution of Fixed Application

After updating processor.py, the application was run again with the same sample inputs. It displayed the expected outputs.

```
C:\Users\palle\Downloads\L6_L7_03_coffee_shop_order\Lab 7>python processor.p
y
==== Running lookup_drink_code ====
Enter drink code prefix (e.g. LAT, CAP, ESP): LAT
Base Price: $ 4.5

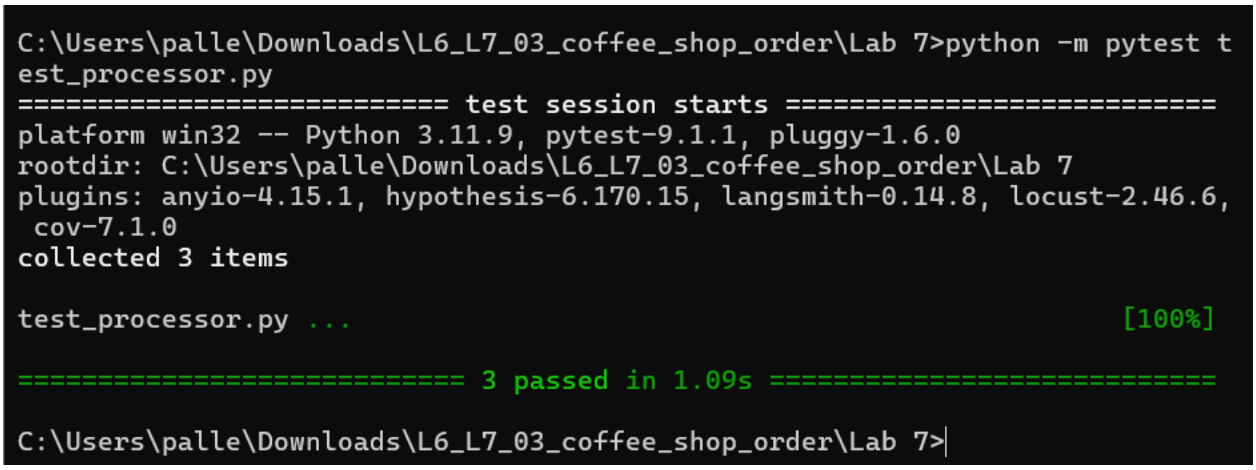
==== Running calculate_average_shot_cost ====
Enter CSV of shot prices (e.g. 1.0, 1.5, 2.0): 1.0, 1.5, 2.0
Average Shot Price: $ 1.5

==== Running parse_order_modifiers ====
Enter modifiers (e.g. milk:oat;syrup:vanilla): milk:oat;syrup:vanilla
Parsed Modifiers: {'milk': 'oat', 'syrup': 'vanilla'}

C:\Users\palle\Downloads\L6_L7_03_coffee_shop_order\Lab 7>
```

7. Fuzz Testing – Successful Test Cases

The Hypothesis tests were rerun after the fixes. All three tests passed successfully, as shown in the terminal output.



8. Conclusion

This lab demonstrated how fuzz testing can reveal problems caused by unexpected inputs. Hypothesis identified edge cases that were not covered by normal sample inputs. After adding input validation and defensive handling to the three functions, all three tests passed.

Deliverables: Lab report, fixed processor.py and updated test_processor.py.